CS 225

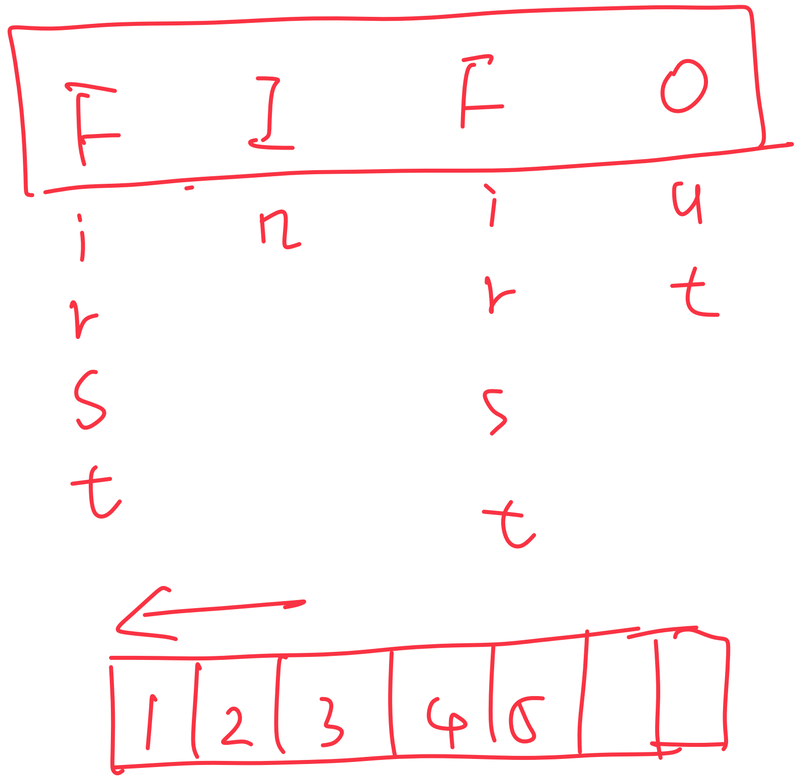
Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16
17    private:
18
19
20  };
21
22  #endif
```



Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16
17    private:
18
19
20  };
21
22  #endif
```

FIFO

void enqueue (QE e); add element to Queue.

Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16
17    private:
18
19
20  };
21
22  #endif
```

FIFO

void enqueue(QE e); add element to Queue.

QE& dequeue( );

Queue.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7
8
9
10
11
12
13
14
15
16
17   private:
18
19
20 };
21
22 #endif
```

void enqueue(QE e); add element to Queue.

QE& dequeue( );

bool isEmpty( );

Queue( );

FIFO

Queue.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7
8
9
10
11
12
13
14
15
16
17   private:
18
19
20 };
21
22 #endif
```

void enqueue(QE e); add element to Queue.

QE& dequeue( );

bool isEmpty( );

Queue( );

rule of three: ~Queue( );

Queue(const Queue&)

Operator=( );

FIFO

Queue.h

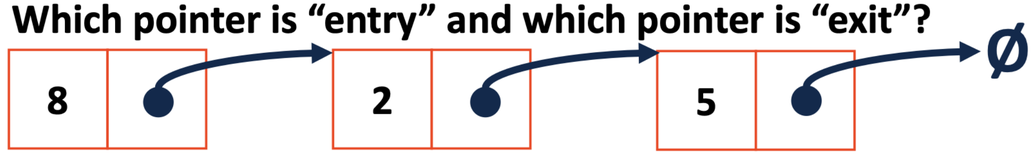
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      struct QueueNode {
13        QE data;
14        QueueNode *next;
15        QueueNode(QE data) ... {}
16      }
17      QueueNode *entry_, *exit_;
18      int size_;
19
20  };
21
22  #endif
```

What type of implementation is this Queue?

Linked List ; Array

How is the data stored on this Queue?

Which pointer is "entry" and which pointer is "exit"?



What is the running time of enqueue()?

What is the running time of dequeue()?

Queue.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7      void enqueue(QE e);
8      QE dequeue();
9      bool isEmpty();
10
11   private:
12     struct QueueNode {
13       QE data;
14       QueueNode *next;
15       QueueNode(QE data) ... {}
16     }
17     QueueNode *entry_, *exit_;
18     int size_;
19
20 };
21
22 #endif
```

What type of implementation is this Queue?

Linked List ; Array

How is the data stored on this Queue?

Storage by value (make a copy)

Which pointer is "entry" and which pointer is "exit"?

What is the running time of enqueue()?

What is the running time of dequeue()?

Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      struct QueueNode {
13        QE data;
14        QueueNode *next;
15        QueueNode(QE data) ... {}
16      }
17      QueueNode *entry_, *exit_;
18      int size_;
19
20  };
21
22  #endif
```

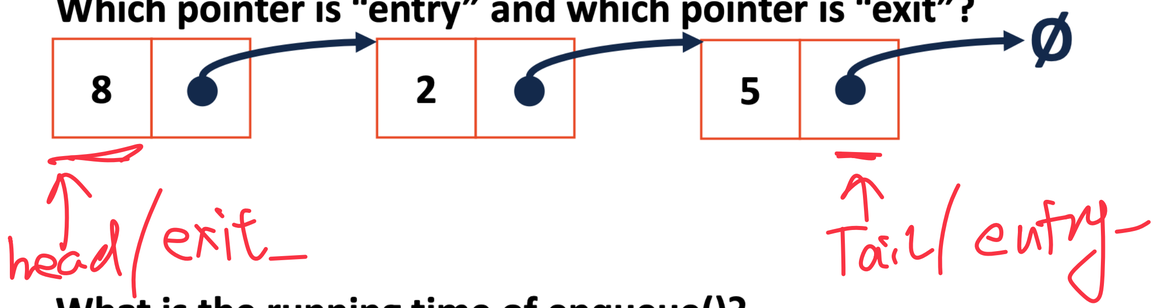
What type of implementation is this Queue?

Linked List ; Array

How is the data stored on this Queue?

Storage by value (make a copy)

Which pointer is "entry" and which pointer is "exit"?



What is the running time of enqueue()?

What is the running time of dequeue()?

Queue.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7      void enqueue(QE e);
8      QE dequeue();
9      bool isEmpty();
10
11   private:
12     struct QueueNode {
13       QE data;
14       QueueNode *next;
15       QueueNode(QE data) ... {}
16     }
17     QueueNode *entry_, *exit_;
18     int size_;
19
20 };
21
22 #endif
```

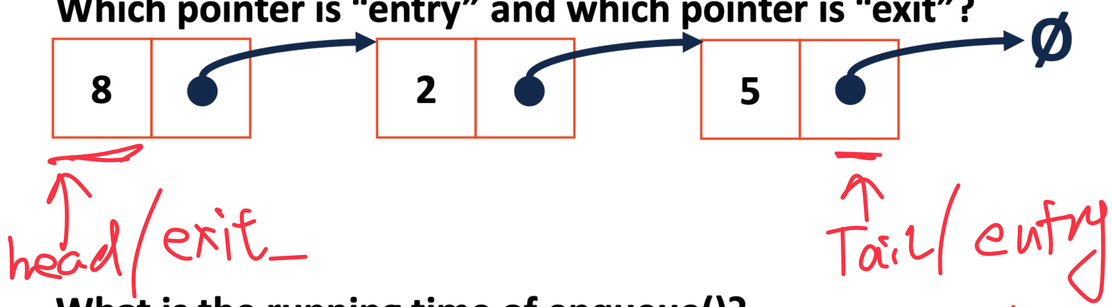
What type of implementation is this Queue?

Linked List ; Array

How is the data stored on this Queue?

Storage by value (make a copy)

Which pointer is "entry" and which pointer is "exit"?



What is the running time of enqueue()?

entry_->next = new QueueNode(...);

entry_ = entry_->next ;

What is the running time of dequeue()?

Queue.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7      void enqueue(QE e);
8      QE dequeue();
9      bool isEmpty();
10
11   private:
12     struct QueueNode {
13       QE data;
14       QueueNode *next;
15       QueueNode(QE data) ... {}
16     }
17     QueueNode *entry_, *exit_;
18     int size_;
19
20 };
21
22 #endif
```

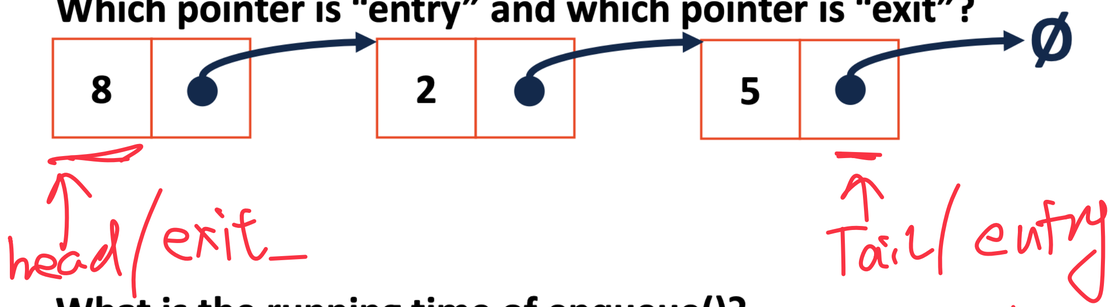
What type of implementation is this Queue?

Linked List ; Array

How is the data stored on this Queue?

Storage by value (make a copy)

Which pointer is "entry" and which pointer is "exit"?



What is the running time of enqueue()?

entry_->next = new QueueNode(...);

entry_ = entry_->next ;

O(1)

What is the running time of dequeue()?

O(1)

Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15
16
17
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)

```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15
16
17
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)

move all data to the left by one location??

```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

Queue.h

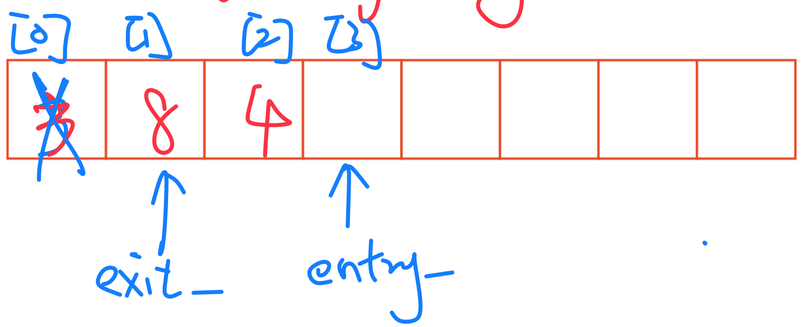
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15      unsigned entry_;
16
17      unsigned exit_;
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)




```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

Queue.h

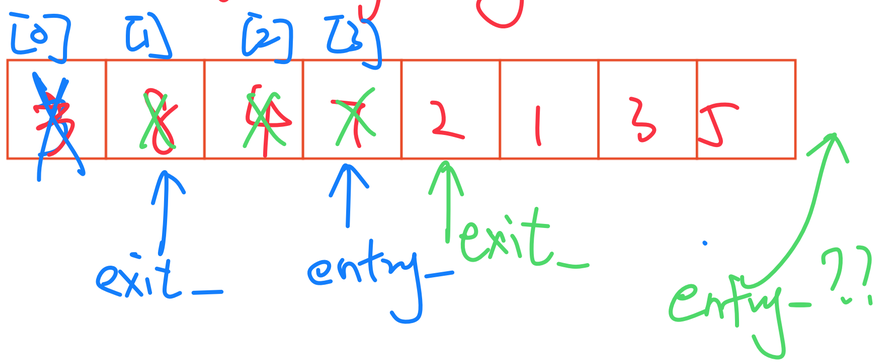
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15      unsigned entry_;
16
17      unsigned exit_;
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)




```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

Queue.h

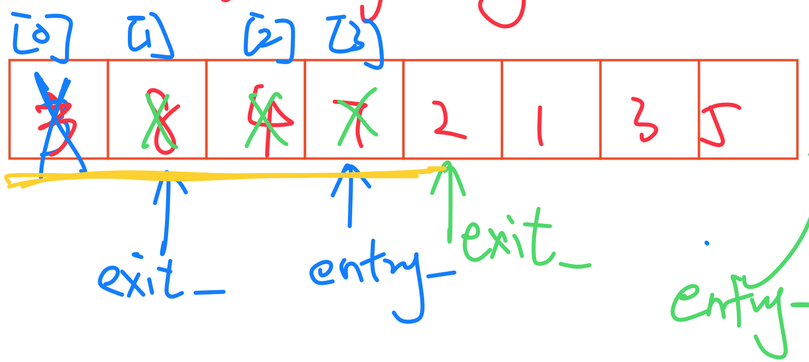
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15      unsigned entry_;
16
17      unsigned exit_;
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)




```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

really??

run out of space??

How to fix??

Queue.h

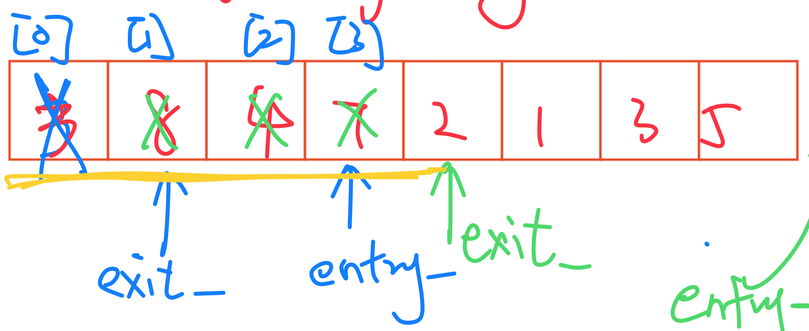
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15      unsigned entry_;
16
17      unsigned exit_;
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)



entry_ ??

nextEntry = (entry_ + 1) % capacity_;

```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

really??

run out of space??

How to fix??

Queue.h

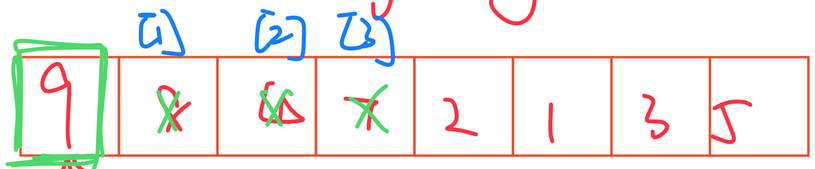
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      void enqueue(QE e);
 8      QE dequeue();
 9      bool isEmpty();
10
11    private:
12      QE *items_;
13      unsigned capacity_;
14      unsigned count_;
15      unsigned entry_;
16
17      unsigned exit_;
18  };
19
20  #endif
21
22
```

What type of implementation is this Queue?

Array

How is the data stored on this Queue?

Storage by value (copy data)



nextEntry = (entry_ + 1) % capacity_;

```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

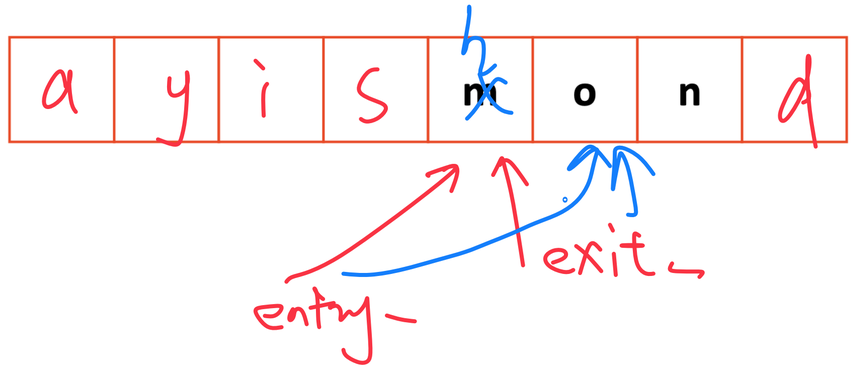
really??

run out of space??

How to fix??

Queue.h

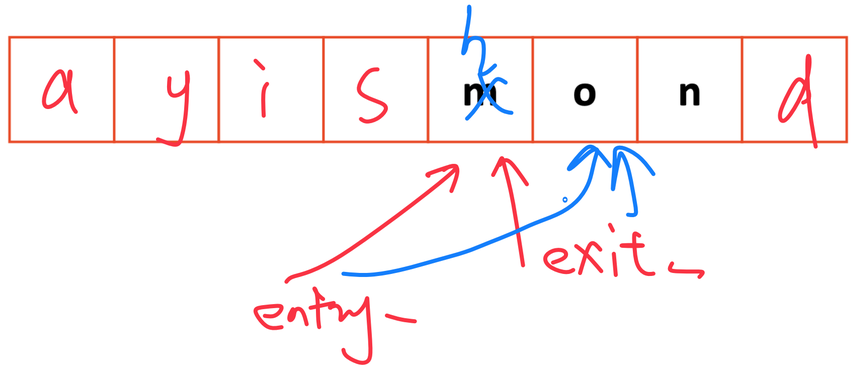
```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      Queue(); // …etc…
 8      void enqueue(QE e);
 9      QE dequeue();
10      bool isEmpty();
11
12    private:
13      QE *items_;
14      unsigned capacity_;
15      unsigned count_;
16      unsigned entry_;
17      unsigned exit_;
18
19  };
20
21  #endif
22
```




```
Queue<char> q;
q.enqueue(m);
q.enqueue(o);
q.enqueue(n);
…
q.enqueue(d);
q.enqueue(a);
q.enqueue(y);
q.enqueue(i);
q.enqueue(s);
q.dequeue();
q.enqueue(h);
q.enqueue(a);
```

Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      Queue(); // …etc…
 8      void enqueue(QE e);
 9      QE dequeue();
10      bool isEmpty();
11
12    private:
13      QE *items_;
14      unsigned capacity_;
15      unsigned count_;
16      unsigned entry_;
17      unsigned exit_;
18
19  };
20
21  #endif
22
```




```
Queue<char> q;
q.enqueue(m);
q.enqueue(o);
q.enqueue(n);
…
q.enqueue(d);
q.enqueue(a);
q.enqueue(y);
q.enqueue(i);
q.enqueue(s);
q.dequeue();
q.enqueue(h);
q.enqueue(a);
```

if (count_ == capacity_) {

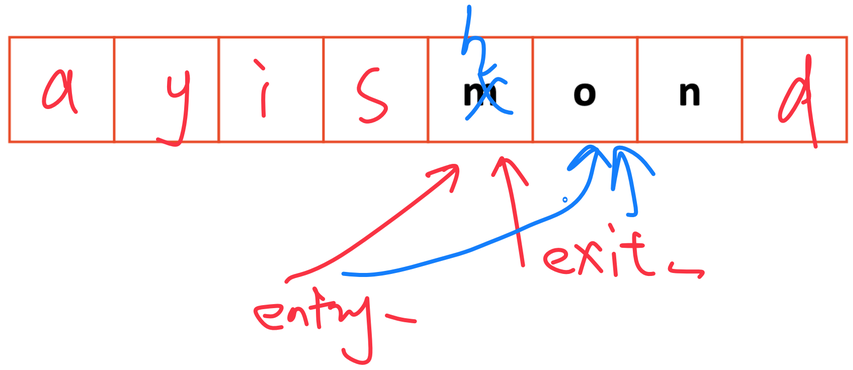
resize!?

no space

}

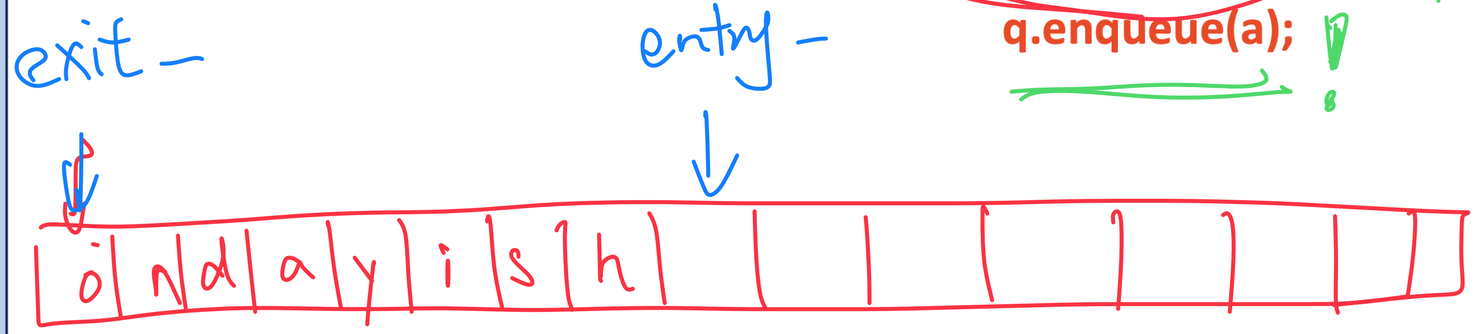
Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      Queue(); // …etc…
 8      void enqueue(QE e);
 9      QE dequeue();
10      bool isEmpty();
11
12    private:
13      QE *items_;
14      unsigned capacity_;
15      unsigned count_;
16      unsigned entry_;
17      unsigned exit_;
18
19  };
20
21  #endif
22
```



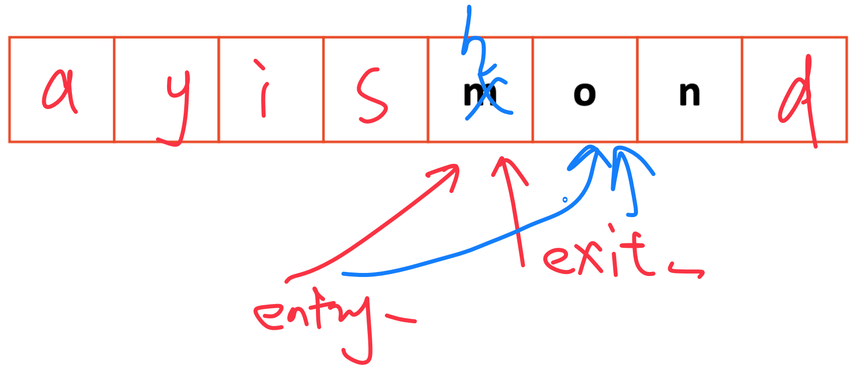

```
Queue<char> q;
q.enqueue(m);
q.enqueue(o);
q.enqueue(n);
…
q.enqueue(d);
q.enqueue(a);
q.enqueue(y);
q.enqueue(i);
q.enqueue(s);
q.dequeue();
q.enqueue(h);
q.enqueue(a);
```

no space



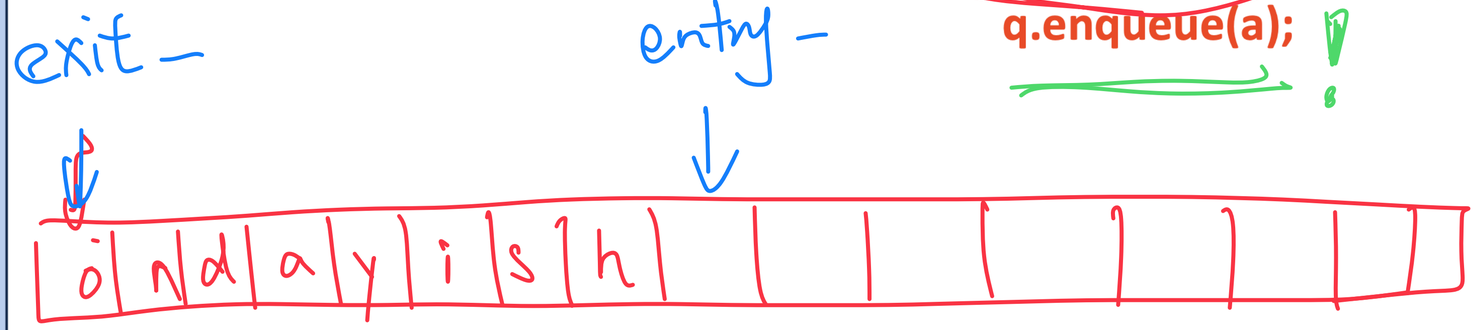
Queue.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7      Queue(); // …etc…
 8      void enqueue(QE e);
 9      QE dequeue();
10      bool isEmpty();
11
12    private:
13      QE *items_;
14      unsigned capacity_;
15      unsigned count_;
16      unsigned entry_;
17      unsigned exit_;
18
19  };
20
21  #endif
22
```




```
Queue<char> q;
q.enqueue(m);
q.enqueue(o);
q.enqueue(n);
…
q.enqueue(d);
q.enqueue(a);
q.enqueue(y);
q.enqueue(i);
q.enqueue(s);
q.dequeue();
q.enqueue(h);
q.enqueue(a);
```

no space



running time ??

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you", bool big = true) ;
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);   // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21      std::cout << (*it).name << " " << (*it).food << std::endl;
22    }
23
24    return 0;
25  }
```

Animal(name, food);

Animal();

Animal(string name); Animal(....);

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you", bool big = true) ;
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);   // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21      std::cout << (*it).name << " " << (*it).food << std::endl;
22    }
23
24    return 0;
25  }
```

Animal(name, food);

Animal();

Animal(string name); Animal(....);

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you", bool big = true) ;
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);   // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21      std::cout << (*it).name << " " << (*it).food << std::endl;
22    }
23
24    return 0;
25  }
```

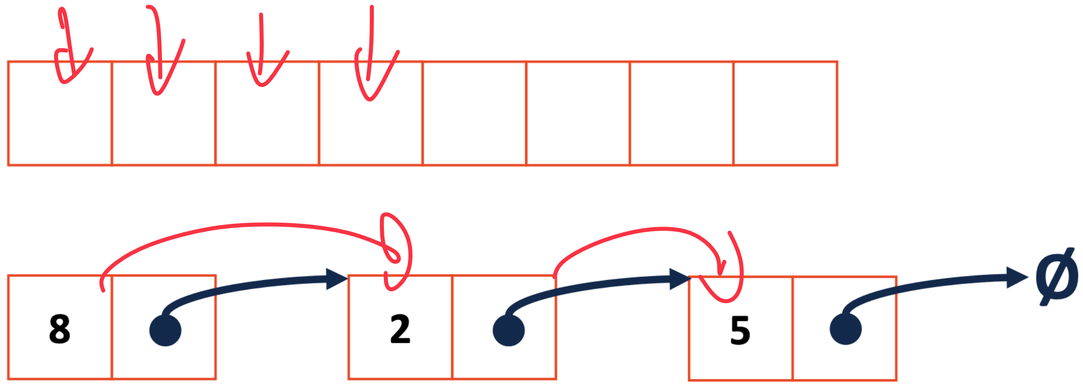
Animal(name, food);

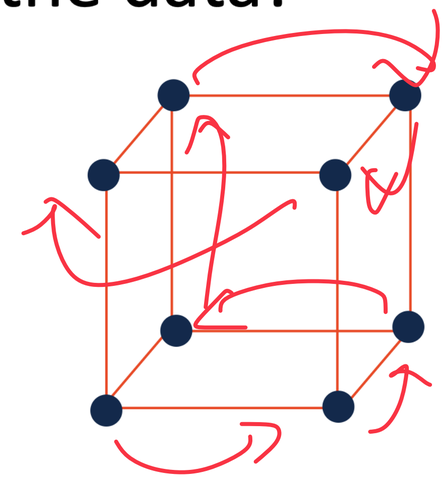
Animal();

Animal(string name); Animal(....);

Iterators

Iterators give client code access to traverse the data!





Operators:

operator *

operator ++

operator == !=

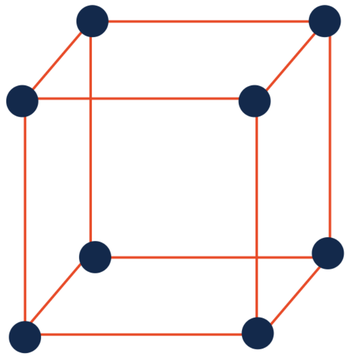
Types of iterators:

Foward

Backwark

Bi-direction.

Iterators encapsulated access to our data:



private var	++	*
listNode *node	node->next	node->data
int index	Index++	arr[index]

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you", bool big = true) ;
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);    // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21      std::cout << (*it).name << " " << (*it).food << std::endl;
22    }
23
24    return 0;
25  }
```

Animal(name, food);

Animal();

Animal(string name); Animal(....);

type

name

indirection

animal

giraffe leaves

penguin fish

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you", bool big = true) ;
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);    // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21      std::cout << (*it).name << " " << (*it).food << std::endl;
22    }
23
24    return 0;
25  }
```

Animal(name, food);

Animal();

Animal(string name); Animal(....);

type

name

indirection

animal

giraffe leaves

penguin fish

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you", bool big = true) ;
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);    // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21      std::cout << (*it).name << " " << (*it).food << std::endl;
22    }
23
24    return 0;
25  }
```

Animal(name, food);

Animal();

Animal(string name); Animal(....);

type

name

one past the last element

indirection

animal

giraffe leaves

penguin fish

bear, you

Iterators

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    // Declaring a vector
    vector<int> v = { 1, 2, 3 };

    // Declaring an iterator
    vector<int>::iterator i;

    int j;

    cout << "Without iterators = ";

    // Accessing the elements without using iterators
    for (j = 0; j < 3; ++j)
    {
        cout << v[j] << " ";
    }

    cout << "\nWith iterators = ";

    // Accessing the elements using iterators
    for (i = v.begin(); i != v.end(); ++i)
    {
        cout << *i << " ";
    }
```

```
// Adding one more element to vector
v.push_back(4);

cout << "\nWithout iterators = ";

// Accessing the elements without using iterators
for (j = 0; j < 4; ++j)
{
    cout << v[j] << " ";
}

cout << "\nWith iterators = ";

// Accessing the elements using iterators
for (i = v.begin(); i != v.end(); ++i)
{
    cout << *i << " ";
}

return 0;
```

```
Without iterators = 1 2 3
With iterators = 1 2 3
Without iterators = 1 2 3 4
With iterators = 1 2 3 4
```

Iterators

Pointers	Iterators
A pointer hold an address in memory.	An iterator may hold a pointer, but it may be something much more complex. For example, an iterator can iterate over data that's on file system, spread across many machines, or generated locally in a programmatic fashion. A good example is an iterator over linked list, the iterator will move through elements that are at nodes in the list whose addresses in RAM may be scattered.
We can perform simple arithmetic on pointers like increment, decrement, add an integer etc.	Not all iterators allow these operations, e.g., we cannot decrement a forward-iterator, or add an integer to a nonrandom-access iterator.
A pointer of type T* can point to any type T object.	An iterator is more restricted, e.g., a vector::iterator can only refer to doubles that are inside a vector container.
We can delete a pointer using **delete**	Since an iterator refers to objects in a container, unlike pointers, there's no concept of **delete** for an iterator. (The container is responsible for memory management.)

Whenever you need to iterate over a standard container, use an iterator over a pointer if supported.

Iterators vs Pointers: A ChatGPT Perspective

Pointers and iterators are both used to navigate through data structures in C++. However, there are some key differences between the two:

1. Type safety: Iterators are generally considered to be safer than pointers because they are type-safe. This means that when you declare an iterator, you must specify the type of data that it will be pointing to. This helps prevent errors that can occur when using pointers, such as accidentally trying to access memory that is outside the bounds of an array.
2. Abstraction: Iterators provide a level of abstraction that pointers do not. This means that iterators can be used to navigate through different types of data structures, such as arrays, vectors, and lists, without having to worry about the underlying implementation details.
3. Range checking: Some iterators provide range checking, which can help prevent errors that can occur when using pointers. For example, if you try to access an element of an array using a pointer that is out of bounds, your program may crash. However, if you use an iterator that provides range checking, it will throw an exception or return an error code instead.
4. Pointer arithmetic: Pointers allow you to perform arithmetic operations on them, such as incrementing or decrementing their values. Iterators, on the other hand, do not allow you to perform arbitrary arithmetic operations on them. Instead, you can only use iterator-specific functions to move them through the data structure.

Overall, while pointers and iterators have some similarities, iterators are generally considered to be safer and more abstract than pointers. However, there are some cases where pointers may still be useful, such as when working with low-level code or interfacing with C-style APIs.

Regenerate response

Be careful about it

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

QueueIter.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7
8
9
10
11
12
13
14
15
16   private:
17
18
19
20 };
21
22 #endif
```

class QueueIterator : ... {
public :
private:
};

Where does the iterator go?

In the public block of the class

↳ user needs to be able to

What additional functions are needed for an iterator?

call this operator

QueueIter.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16    private:
17
18
19
20  };
21
22  #endif
```

class QueueIterator : ... {
public : (A)
private: (B)
}; (C)

Where does the iterator go?

In the public block of the class

↳ user needs to be able to call this operator

What additional functions are needed for an iterator?

- begin() ; end() ; ??

function of Iterator

QueueIter.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16    private:
17
18
19
20  };
21
22  #endif
```

class QueueIterator : ... {
public : (A)
private: (B)
}; (C)

Where does the iterator go?

In the public block of the class

↳ user needs to be able to

What additional functions are needed for an iterator?

call this operator

- begin( ) ; end( ) ; ??

function of Iterator

zoo.begin( )

Queue class

QueueIter.h

```
1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <class QE>
5  class Queue {
6    public:
7
8
9
10
11
12
13
14
15
16   private:
17
18
19
20 };
21
22 #endif
```

class QueueIterator : ... {
public : (A)
private: (B)
}; (C)

Where does the iterator go?

In the public block of the class
↳ user needs to be able to

What additional functions are needed for an iterator?

call this operator

- begin() ; end() ; ++ ; ??

function of Iterator ↳ (C)

- operator *
*it ; inside iterator. ↳ (A)

QueueIter.h

```
 1  #ifndef QUEUE_H
 2  #define QUEUE_H
 3
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16    private:
17
18
19
20  };
21
22  #endif
```

class QueueIterator : ... {
public : (A)
private: (B)
}; (C)

Where does the iterator go?

In the public block of the class
↳ user needs to be able to

What additional functions are needed for an iterator?

call this operator

- begin() ; end() ; += ;??

function of Iterator ↳ (C)

- operator *
  *it ; inside iterator. ↳ (A)

- Anything in (B) ??

Queue.h

```
 4  template <class QE>
 5  class Queue {
 6    public:
 7      class QueueIterator : public std::iterator<std::bidirectional_iterator_tag, T> {
 8        public:
 9          QueueIterator(unsigned index);
10          QueueIterator& operator++();
11          bool operator==(const QueueIterator &other);
12          bool operator!=(const QueueIterator &other);
13          QE& operator*();
14          QE* operator->();
15        private:
16          unsigned location_;
17      }
18
19                                  Access to location in the data structure.
20
21
22    private:
23      QE* arr_; unsigned capacity_, count_, entry_, exit_;
24
25                  Array based.
26  };
```

stlList.cpp

```
 1 #include <list>
 2 #include <string>
 3 #include <iostream>
 4
 5 struct Animal {
 6   std::string name, food;
 7   bool big;
 8   Animal(std::string name = "blob", std::string food = "you", bool big = true) :
 9     name(name), food(food), big(big) { /* none */ }
10 }
11
12 int main() {
13   Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14   std::list<Animal> zoo;
15
16   zoo.push_back(g);
17   zoo.push_back(p);   // std::list's insertAtEnd
18   zoo.push_back(b);
19
20   for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
21     std::cout << (*it).name << " " << (*it).food << std::endl;
22   }
23
24   return 0;
25 }
```

queue

Queue<Animal>::QueueIterator

stlList.cpp

```
 1 #include <Queue.h>
 2 #include <string>
 3 #include <iostream>
 4
 5 struct Animal {
 6   std::string name, food;
 7   bool big;
 8   Animal(std::string name = "blob", std::string food = "you", bool big = true) :
 9     name(name), food(food), big(big) { /* none */ }
10 }
11
12 int main() {
13   Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14   Queue<Animal> zoo;
15
16   zoo.push_back(g);
17   zoo.push_back(p);
18   zoo.push_back(b);
19
20   for ( Queue<Animal>::QueueIterator it = zoo.begin(); it != zoo.end(); it++ ) {
21     std::cout << (*it).name << " " << (*it).food << std::endl;
22   }
23
24   return 0;
25 }
```

Function Objects (aka "Functors")

a class with an overloaded "Function call" operator ()

Functors are objects that can be called like a function.

```
1
2
3
4
```

Function Objects (aka "Functors")

a class with an overloaded "Function call" operator ()

Functors are objects that can be called like a function.

```
1  Doubler d;
2  int value;
3  d(value);
4
```

looks like a function!

but this is an object.

```
int Doubler::operator()(int value){

}
```

Function Objects (aka "Functors")

a class with an overloaded "Function call" operator ()

Functors are objects that can be called like a function.

```
1  Doubler d;
2  int value;
3  d(value);
4
```

looks like a function!

but this is an object.

```
int Doubler::operator()(int value){

}
```

Functor

A C++ functor (function object) is a class or struct object that can be called like a function

It overloads the **function-call operator** () and allows us to use an object like a function.

main.cpp

```cpp
#include <iostream>
using namespace std;

class Greet {

  public:
    // overload function call/parentheses operator
    void operator()() {
      cout << "Hello World!";
    }
};

int main() {

  // create an object of Greet class
  Greet greet;

  // call the object as a function
  greet();

  return 0;
}
```

- Create a class Greet, whose object can be called like a function
- Overload the operator()

```cpp
// displays "Hello World!"
greet.operator()();
```

Output

```
/tmp/YaL55ZXsIZ.o
Hello World!
```

Functor - accepts parameters and returns a value

main.cpp

```cpp
#include <iostream>
using namespace std;

class Add {

  public:
    // overload function call operator
    // accept two integer arguments
    // return their sum
    int operator() (int a, int b) {
      return a + b;
    }
};

int main() {

  // create an object of Add class
  Add add;

  // call the add  object
  int sum = add(100, 78);

  cout << "100 + 78 = " << sum;

  return 0;
}
```

- a functor with an integer return type and two integer parameters to find their sum

- Create the add object from Add class
- Call the add object with parameters 100 and 78
- Return the sum of two ints

Output

```
/tmp/kgS4UZulTF.o
100 + 78 = 178
```

Functor - with a member variable

```
#include<iostream>
using namespace std;
class Add_To_Sum {
  private:
    int initial_sum;

  public:

    // constructor to initialize member variable
    Add_To_Sum(int sum) {
      initial_sum = sum;
    }

    // overload function call operator
    int operator()(int num) {
      return initial_sum + num;
    }

};
int main() {
  // create object of Add_To_Sum class
  // initialize member variable of object with value 100
  Add_To_Sum add = Add_To_Sum(100);
  // call the add object with 78 as argument
  int final_sum = add(78);

  cout << "100 + 78 = " << final_sum;
  return 0;
}
```

- Add_To_Sum class with member variable initial_sum that is initialized with a constructor
- Overload operator() with parameter num, and return the sum

- Initialize initial_sum = 100;
- Call the functor with parameter 78
- Return the sum of two ints

Output

```
/tmp/kgS4UZulTF.o
100 + 78 = 178
```

Why Functors

- Functors can contain state, e.g., 100 in the previous example, 100 can be anything
- Pass functors as arguments to other functions such as std::transform or the other standard library algorithms, flexible functors to be more flexible
- …

https://stackoverflow.com/questions/356950/what-are-c-functors-and-their-uses

Doubler.h

```
 1  #ifndef DOUBLER_H
 2  #define DOUBLER_H
 3
 4  class Doubler {
 5    public:
 6      int operator()(int value);
 7
 8
 9
10  };
11
12  #endif
```

Define a state
: how many times it is called

private:
callcount_;

```
1  #include "Doubler.h"
2
3  int Doubler::operator()(int value) {
4
5    return value * 2;
6  }
```

callcounter_++;

Functor in templated function

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3      return modifier(value);
4  }
```

Apply modifier to a value.

```
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

Functor in templafeed function

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3      return modifier(value);
4  }
```

Apply modifier to a value.

```
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

call function modify on value of type int with functor Doubler

Any requirement??

Functor in templafeed function

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3      return modifier(value);
4  }
```

Apply modifier to a value.

```
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

call function modify on value of type int with functor Doubler

Any requirement??

- The type given to template Modifier must be functor

Functor in templated function

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3      return modifier(value);
4  }
```

Apply modifier to a value.

```
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

call function modify on value of type int with functor Doubler

Any requirement??

- The type given to template Modifier must be functor

- The type given to Modifier must have an overload operator()(Value)

Functor in templated function

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3      return modifier(value);
4  }
```

Apply modifier to a value.

modify(string, Doubler) ??

```
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

- The type given to template Modifier must be functor

- The type given to Modifier must have an overload operator()(Value)

Functor in templated function

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3      return modifier(value);
4  }
```

Apply modifier to a value.

```
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

modify(string, Doubler) ?

- Templates are checked at the compile time!!

- The type given to template Modifier must be functor

- The type given to Modifier must have an overload operator()(Value)

```
1  template<class Iter, class Formatter>
2  void print(Iter first, Iter second, Formatter printer) {
3    while ( first != second ) {
4      printer( *first );
5      first++;
6    }
7  }
```

This is a function called print whose inputs are two type Iter ?: and a type Formatter.

This function appears to:

```
1  template<class Iter, class Formatter>
2  void print(Iter first, Iter second, Formatter printer) {
3    while ( first != second ) {
4      printer( *first );
5      first++;
6    }
7  }
```

This is a function called print whose inputs are two type Iter ?: and a type Formatter ??.

This function appears to:

print out the values in an Iterator from first to second.

```
1  template<class Iter, class Formatter>
2  void print(Iter first, Iter second, Formatter printer) {
3    while ( first != second ) {
4      printer( *first );
5      first++;
6    }
7  }
```

pointer??

functor

This is a function called _____ whose inputs are two _____ and a _____.

print

type Iter ??

type Formatter

This function appears to:

print out the values in an Iterator from first to second.

```
1  template<class Iter, class Formatter>
2  void print(Iter first, Iter second, Formatter printer) {
3    while ( first != second ) {
4      printer( *first );
5      first++;
6    }
7  }
```

pointer??

functor

This is a function called _____ whose inputs are two _____ and a _____.

print

type Iter ??

type Formatter

This function appears to:

print out the values in an Iterator from first to second.

2 template classes

stlList.cpp (2)

print<Queue<Animal>::QueueIterator, AnimalPrinter>
(zoo.begin(), zoo.end(), printBig)

```
 1 #include <Queue.h>
 2 #include <string>
 3 #include <iostream>
 4
 5 struct Animal {
 6   std::string name, food;
 7   bool big;
 8   Animal(std::string name = "blob", std::string food = "you", bool big = true) :
 9     name(name), food(food), big(big) { /* none */ }
10 }
11
12 int main() {
13   Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
14   Queue<Animal> zoo;
15
16   zoo.push_back(g);
17   zoo.push_back(p);
18   zoo.push_back(b);
19
20   for ( Queue<Animal>::QueueIterator it = zoo.begin(); it != zoo.end(); it++ ) {
21     std::cout << (*it).name << " " << (*it).food << std::endl;
22   }
23
24   return 0;
25 }
```

stlList.cpp

2 template classes

print<Queue<Animal>::QueueIterator, AnimalPrinter>
(zoo.begin(), zoo.end(), printBig)

printSmall

Generically through linear data structures using different strategies.

```
#include <Queue.h>
#include <string>
#include <iostream>

struct Animal {
  std::string name, food;
  bool big;
  Animal(std::string name = "blob", std::string food = "you", bool big = true) :
    name(name), food(food), big(big) { /* none */ }
}

int main() {
  Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
  Queue<Animal> zoo;

  zoo.push_back(g);
  zoo.push_back(p);
  zoo.push_back(b);

  for ( Queue<Animal>::QueueIterator it = zoo.begin(); it != zoo.end(); it++ ) {
    std::cout << (*it).name << " " << (*it).food << std::endl;
  }

  return 0;
}
```

Queue.h

```
 4  template <class QE>
 5  class Queue {
 6    public:
 7      class QueueIterator : public std::iterator<std::bidirectional_iterator_tag, T> {
 8        public:
 9          QueueIterator(unsigned index);
10          QueueIterator& operator++();
11          bool operator==(const QueueIterator &other);
12          bool operator!=(const QueueIterator &other);
13          QE& operator*();
14          QE* operator->();
15        private:
16          int location_;
17
18      };
19
20
21    /* ... */
22
23    private:
24      QE* arr_; unsigned capacity_, count_, entry_, exit_;
25  };
26
```

Not inherited from Queue (variables).

arr_[location] ??

queue_.arr_[location]

Queue &queue_;

Queue.h

```
 4  template <class QE>
 5  class Queue {
 6    public:
 7      class QueueIterator : public std::iterator<std::bidirectional_iterator_tag, T> {
 8        public:
 9          QueueIterator(unsigned index);
10          QueueIterator& operator++();
11          bool operator==(const QueueIterator &other);
12          bool operator!=(const QueueIterator &other);
13          QE& operator*();
14          QE* operator->();
15        private:
16          int location_;
17
18      };
19
20
21    /* ... */
22
23    private:
24      QE* arr_; unsigned capacity_, count_, entry_, exit_;
25  };
26
```

Not inherited from Queue (variables).

arr_[location] ??

queue_.arr_[location]

Queue &queue_;

Big Ideas

Member functions and variables are only inherited in derived classes.

Class scope determines access to private members.

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

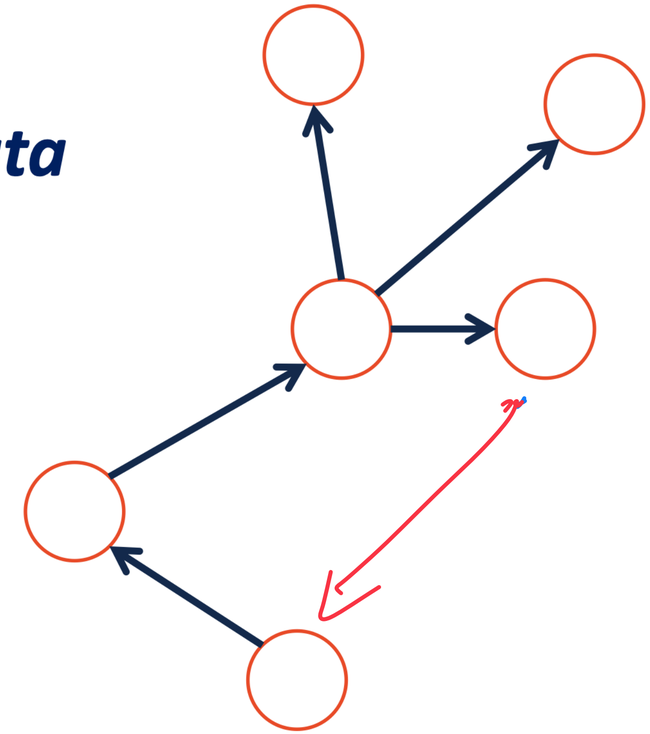
Trees

"The most important non-linear data structure in computer science."

- David Knuth, The Art of Programming, Vol. 1

A tree is:

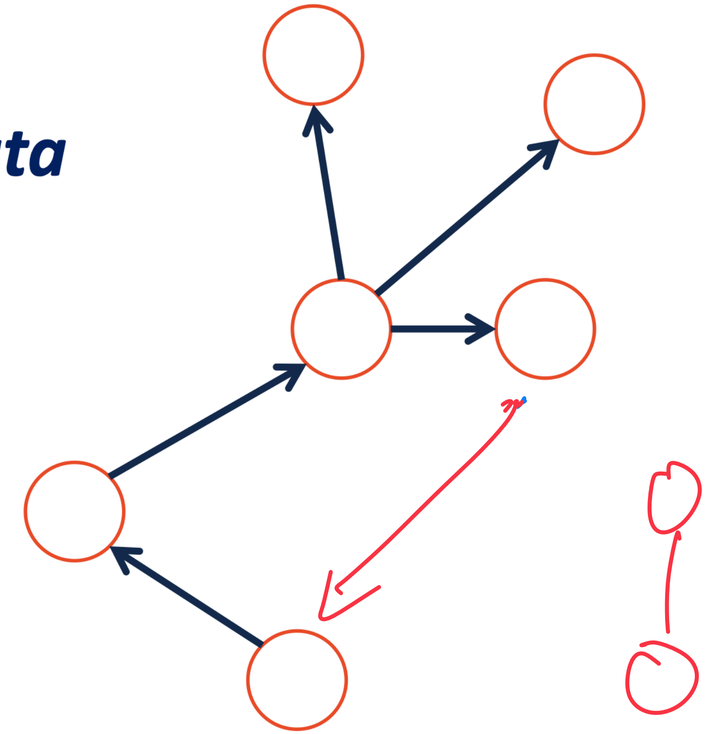
- Acyclic Graph
-



Trees

"The most important non-linear data structure in computer science."

- David Knuth, The Art of Programming, Vol. 1



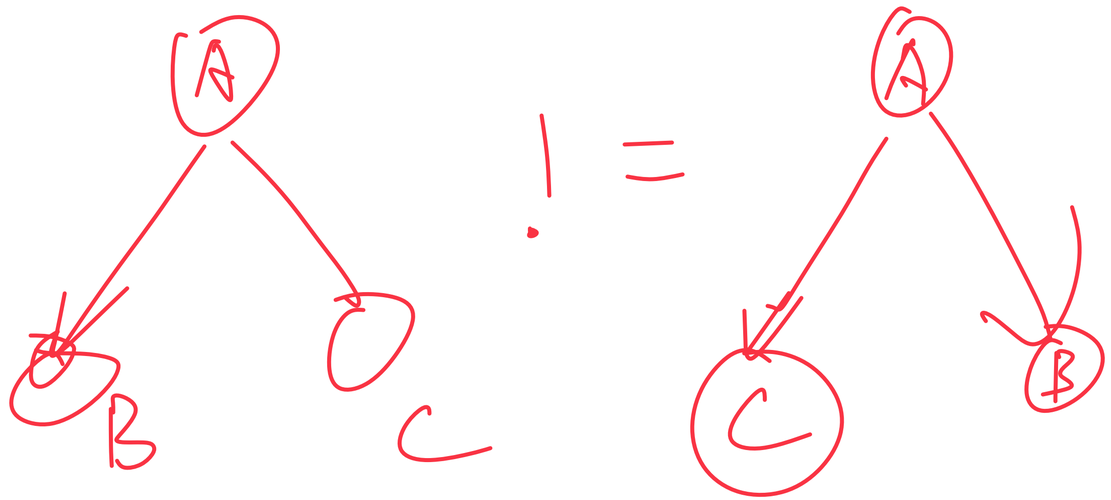
A tree is:

- Acyclic Graph
- Connected graph → No Disjoint

More Specific Trees

We'll focus on a specific type of trees:

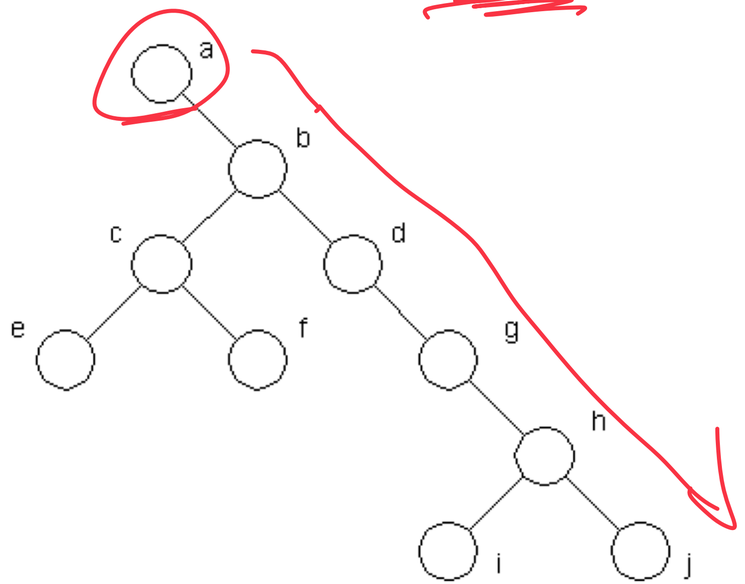
- Rooted
- Ordered
- Directed



Tree Terminology

vertex / node

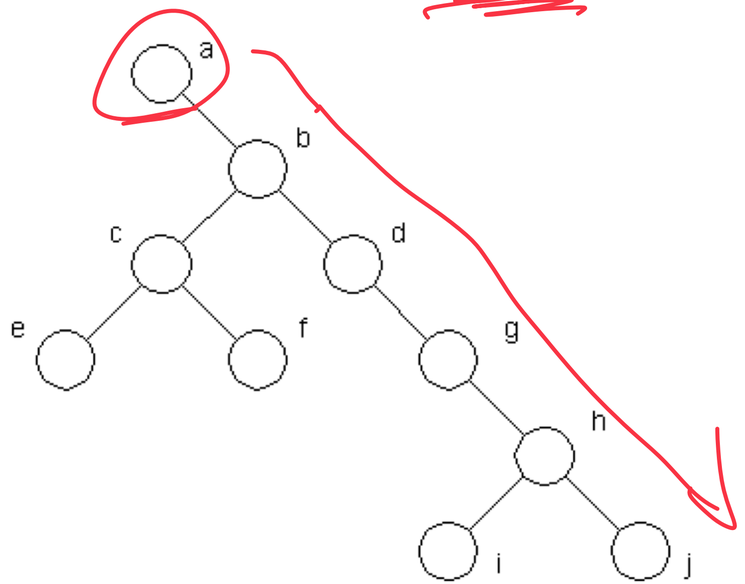
- What's the longest "word" you can make using the vertex labels in the tree (repeats allowed)?
- Find an edge that is not on the longest path in the tree. Give that edge a reasonable name.
- One of the vertices is called the root of the tree. Which one? a
- Make an "word" containing the names of the vertices that have a parent but no sibling.
- How many parents does each vertex have?
- Which vertex has the fewest children?
- Which vertex has the most ancestors?
- Which vertex has the most descendants?
- List all the vertices is b's left subtree.
- List all the leaves in the tree.



Tree Terminology

vertex / node

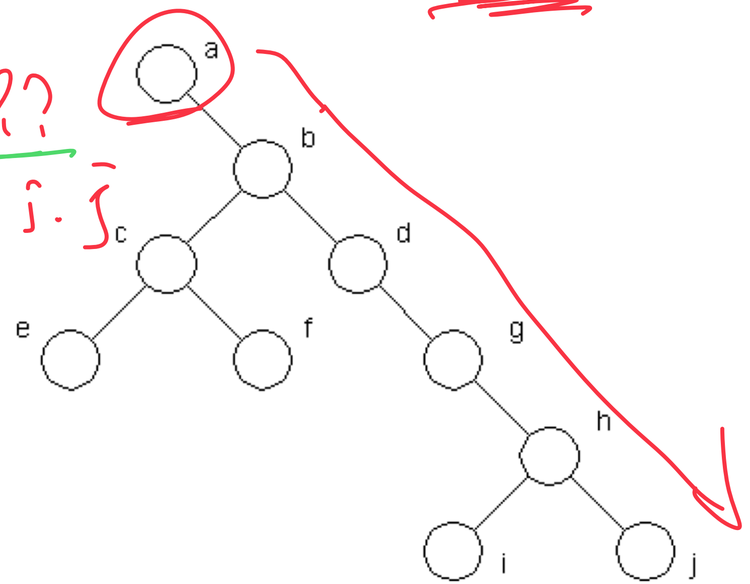
- What's the longest "word" you can make using the vertex labels in the tree (repeats allowed)?
- Find an edge that is not on the longest path in the tree. Give that edge a reasonable name.
- One of the vertices is called the root of the tree. Which one? a
- Make an "word" containing the names of the vertices that have a parent but no sibling.
- How many parents does each vertex have?
- Which vertex has the fewest children?
- Which vertex has the most ancestors?
- Which vertex has the most descendants?
- List all the vertices is b's left subtree.
- List all the leaves in the tree.



Tree Terminology

vertex / node

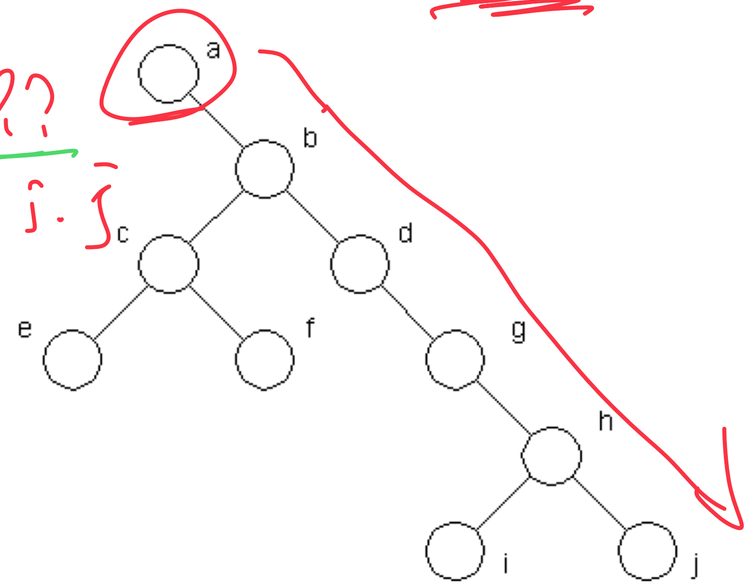
- What's the longest "word" you can make using the vertex labels in the tree (repeats allowed)?
- Find an edge that is not on the longest path in the tree. Give that edge a reasonable name.
- One of the vertices is called the root of the tree. Which one? a
- Make an "word" containing the names of the vertices that have a parent but no sibling. b, g, h.
- How many parents does each vertex have? 1 ??
- Which vertex has the fewest children? e, f, i, j
- Which vertex has the most ancestors? i, j
- Which vertex has the most descendants? a
- List all the vertices is b's left subtree.
- List all the leaves in the tree.



Tree Terminology

vertex / node

- What's the longest "word" you can make using the vertex labels in the tree (repeats allowed)?
- Find an edge that is not on the longest path in the tree. Give that edge a reasonable name.
- One of the vertices is called the root of the tree. Which one? a
- Make an "word" containing the names of the vertices that have a parent but no sibling. b, g, h.
- How many parents does each vertex have? 1 ??
- Which vertex has the fewest children? e, f, i, j
- Which vertex has the most ancestors? i, j
- Which vertex has the most descendants? a.
- List all the vertices is b's left subtree. {c, e, f}
- List all the leaves in the tree. {e, f, i, j}.



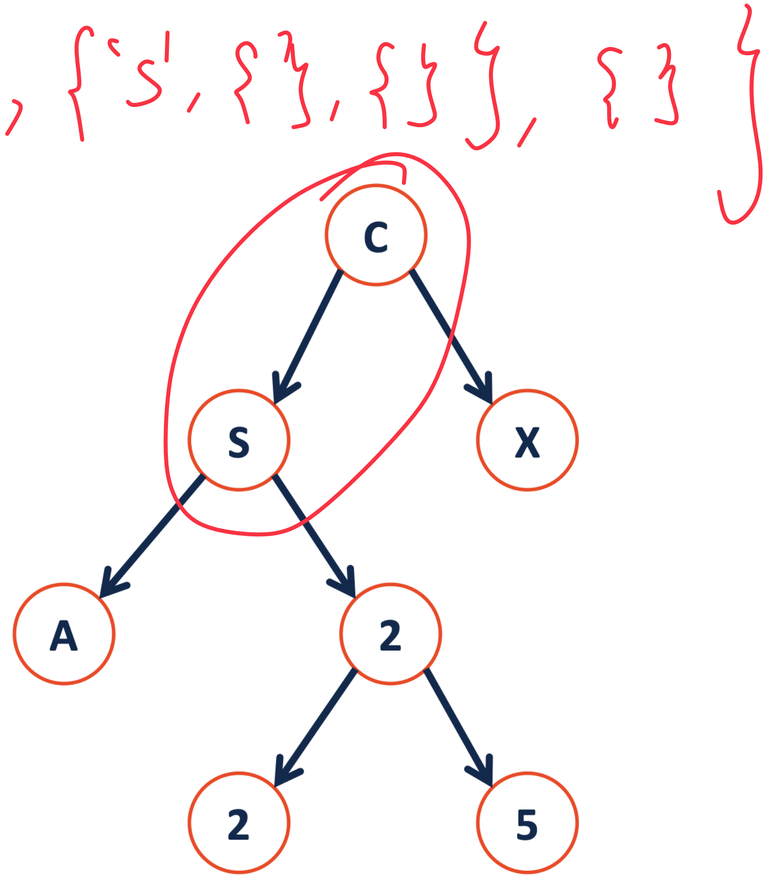
Binary Tree – Defined

A *binary tree* T is either:

- **T={}**

OR

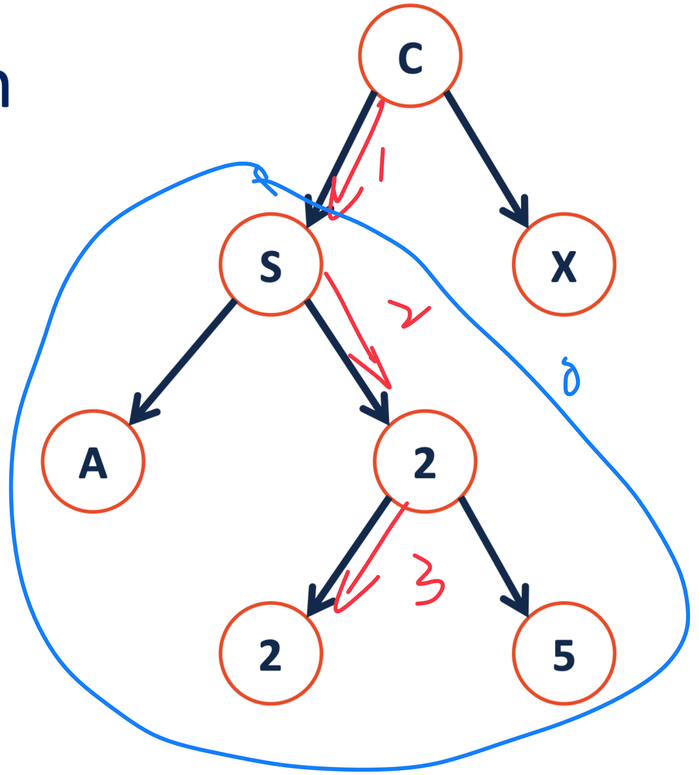
- **T={r, T_L, T_R}**



Tree Property: height

***height(T)*:** length of the longest path from the root to a leaf

↳ count Edges



Given a binary tree T:

***height(T)* =**

| if **{r, T_L, T_R}**, max(height(T_L), height(T_R)) + 1

|

| if {}, -1

T = 2+1

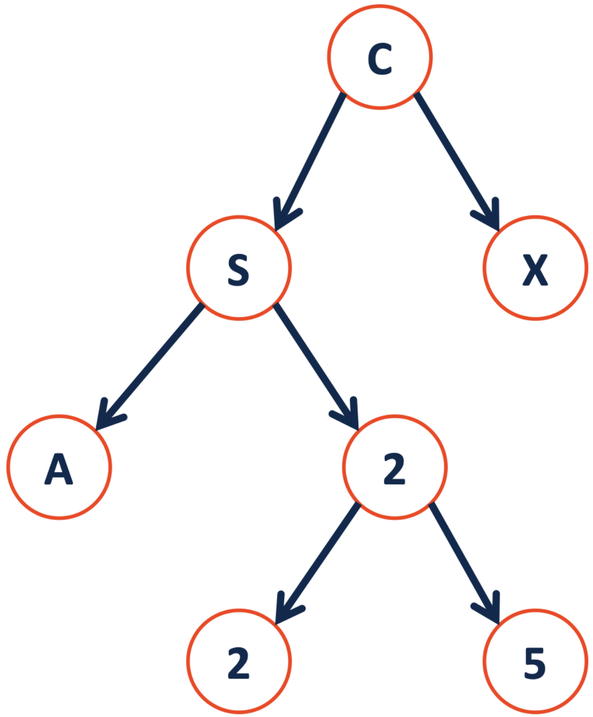
Tree Property: full

A tree ***F*** is **full** if and only if:

1. **F={}**

OR

2. **F={r,** T_L**,** T_R**}** where T_L and T_R are both empty or both not empty



Tree Property: perfect

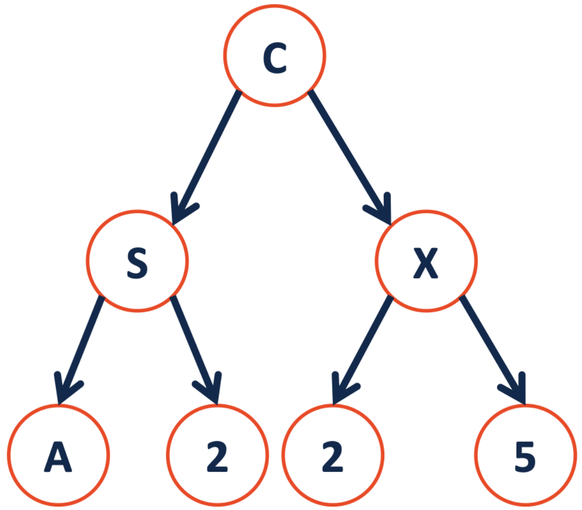
A **perfect** tree ***P*** is:

$P_h = 2^{h+1} - 1$

1. **P={}**

OR

2. **P={r, T_L, T_R}** where T_L and T_R are of height T_{h-1}



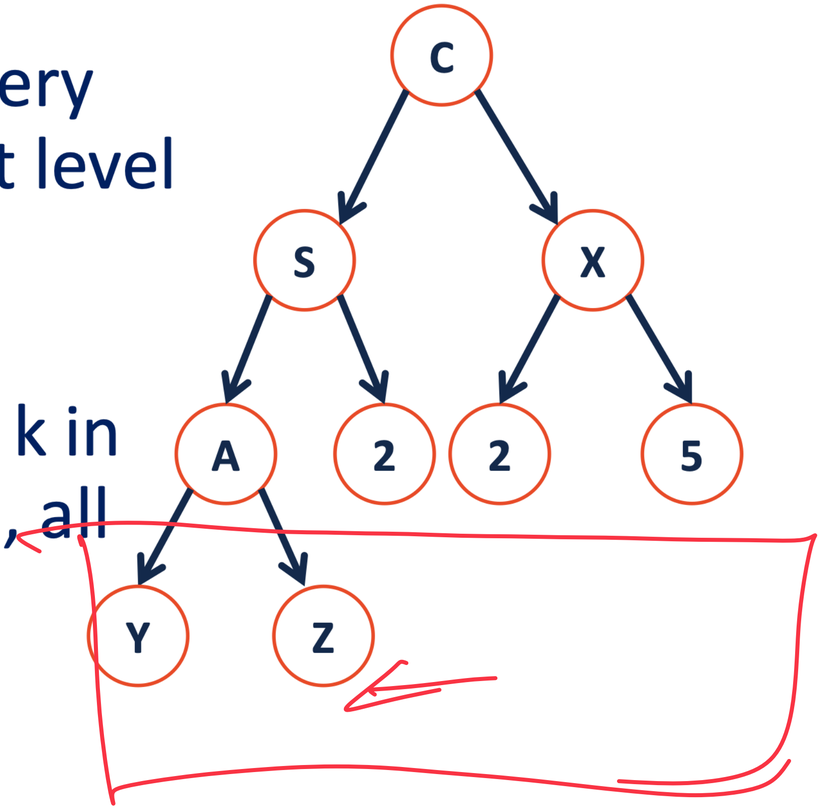
$P_0 : \{ 'c', \{\}, \{\} \}$

$P_1 : \{ 'c', \{ 's', \{\}, \{\} \}, \{ 'x', \{\}, \{\} \} \}$

Tree Property: complete

Conceptually: A perfect tree for every level except the last, where the last level is “pushed to the left”.

Slightly more formal: For any level k in [0, h-1], k has 2^k nodes. For level h, all nodes are “pushed to the left”.



Tree Property: complete

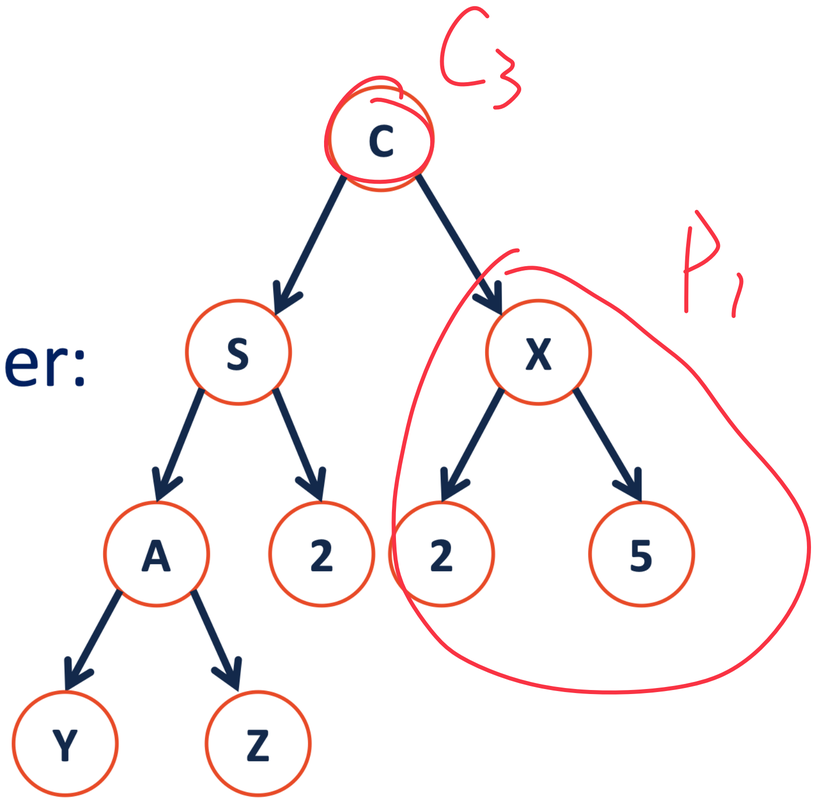
A **complete** tree ***C*** of height **h**, $\mathbf{C_h}$:

1. $\mathbf{C_{-1}} = \{\}$
2. $\mathbf{C_h}$ *(where h>0)* $= \mathbf{\{r, T_L, T_R\}}$ and either:

T_L is $\mathbf{C_{h-1}}$ and T_R is $\mathbf{P_{h-2}}$

OR

T_L is $\mathbf{P_{h-1}}$ and T_R is $\mathbf{C_{h-1}}$



Tree Property: complete

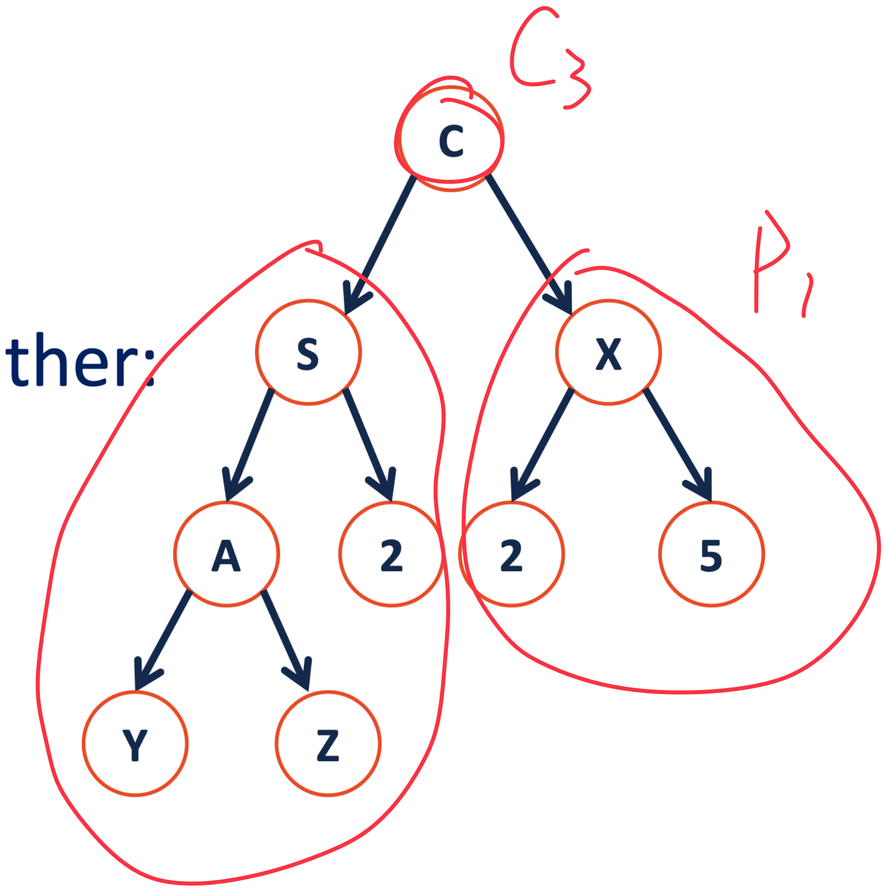
A **complete** tree ***C*** of height **h**, $\mathbf{C_h}$:

1. $\mathbf{C_{-1}} = \{\}$
2. $\mathbf{C_h}$ *(where h>0)* $= \mathbf{\{r, T_L, T_R\}}$ and either:

T_L is $\mathbf{C_{h-1}}$ and T_R is $\mathbf{P_{h-2}}$

OR

T_L is $\mathbf{P_{h-1}}$ and T_R is $\mathbf{C_{h-1}}$



Tree Property: complete

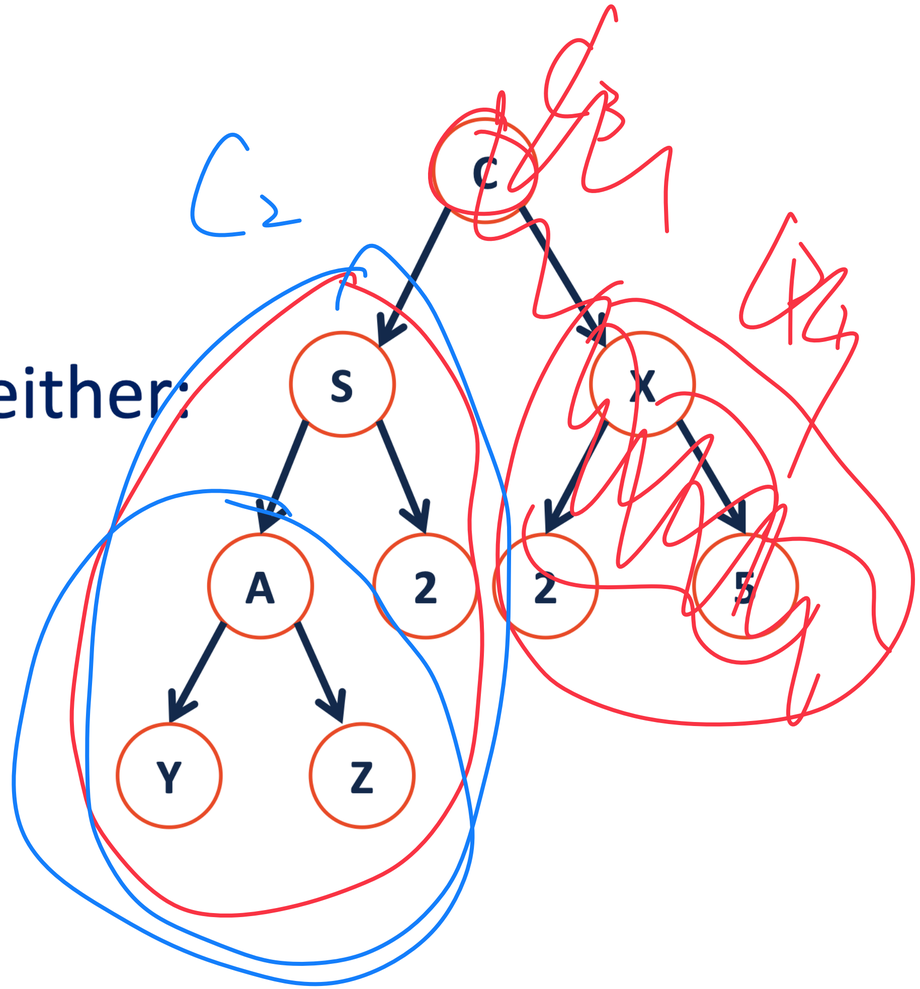
A **complete** tree ***C*** of height **h**, $\mathbf{C_h}$:

1. $\mathbf{C_{-1}} = \{\}$
2. $\mathbf{C_h}$ *(where h>0)* $= \mathbf{\{r, T_L, T_R\}}$ and either:

T_L is $\mathbf{C_{h-1}}$ and T_R is $\mathbf{P_{h-2}}$

OR

T_L is $\mathbf{P_{h-1}}$ and T_R is $\mathbf{C_{h-1}}$



Tree Property: complete

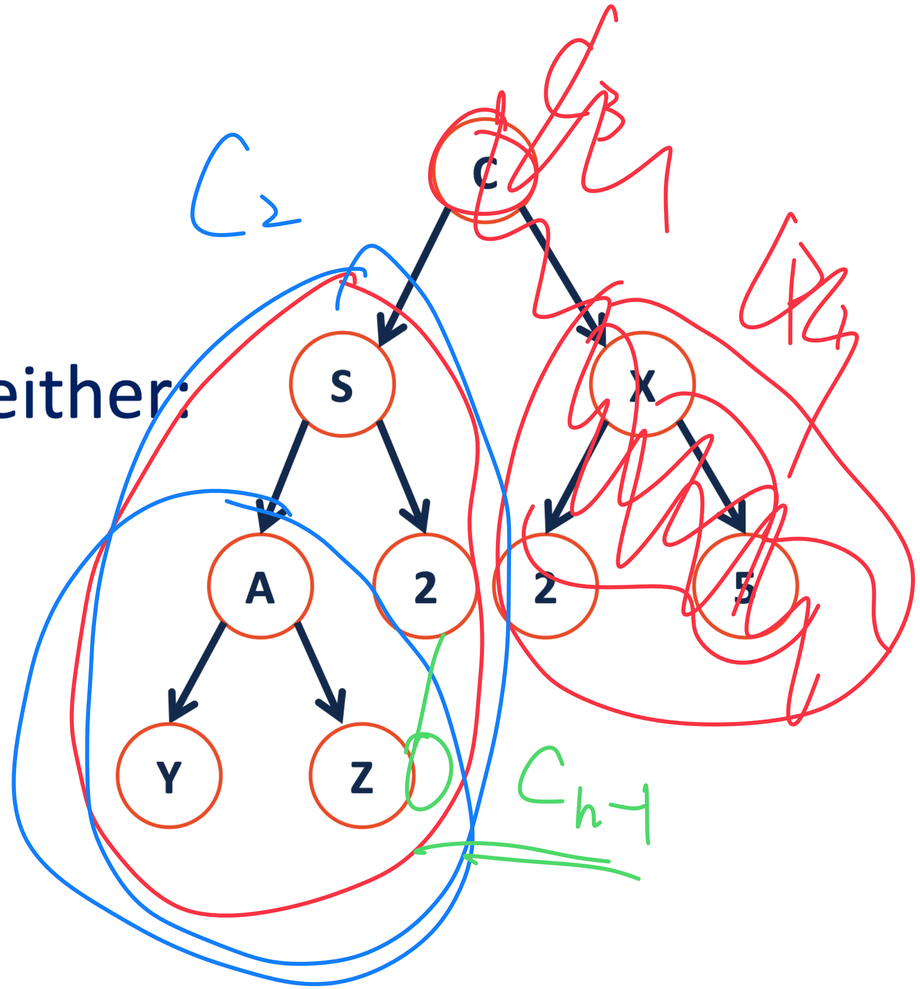
A **complete** tree ***C*** of height **h**, $\mathbf{C_h}$:

1. $\mathbf{C_{-1}} = \{\}$
2. $\mathbf{C_h}$ *(where h>0)* $= \mathbf{\{r, T_L, T_R\}}$ and either:

T_L is $\mathbf{C_{h-1}}$ and T_R is $\mathbf{P_{h-2}}$

OR

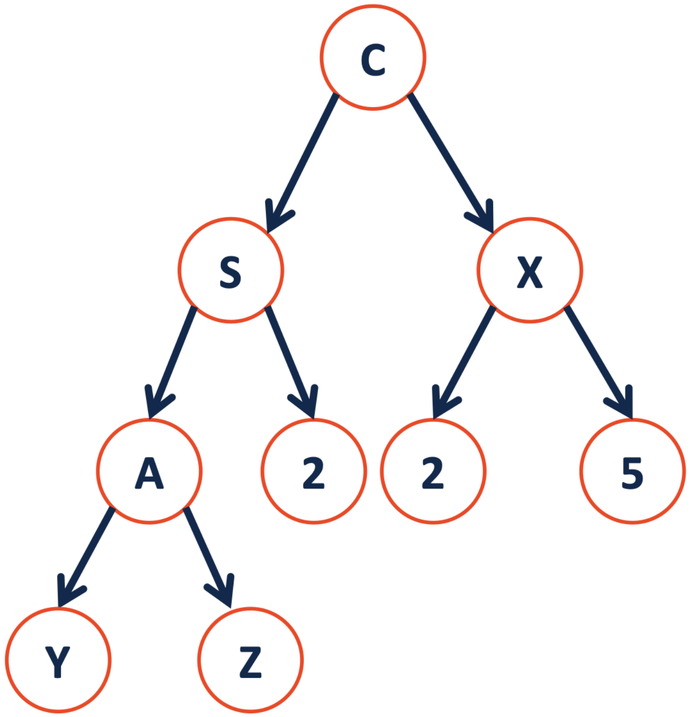
T_L is $\mathbf{P_{h-1}}$ and T_R is $\mathbf{C_{h-1}}$



Tree Property: complete

Is every **full** tree **complete**?

Is every **complete** tree **full**?



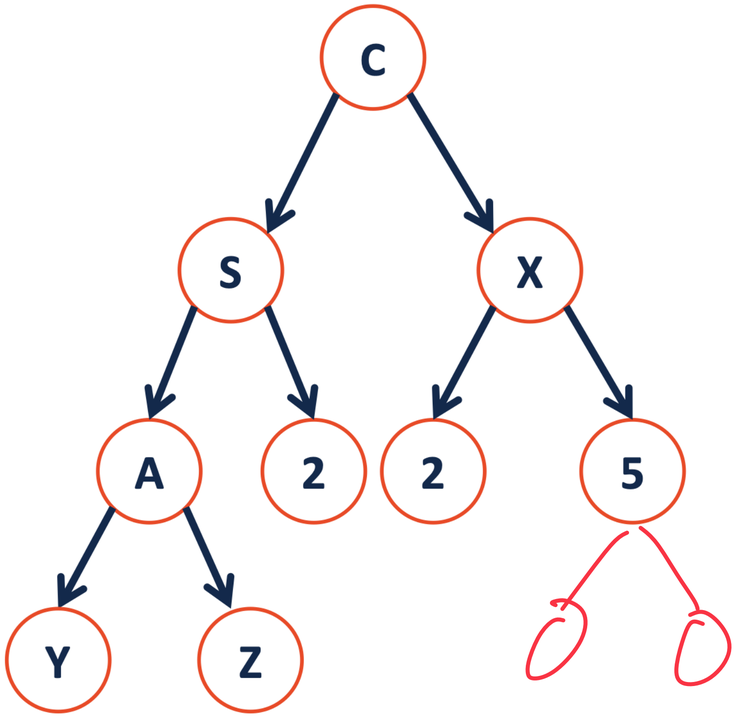
Tree Property: complete

Is every **full** tree **complete**?

Full ?? ✓

Complete ?? X

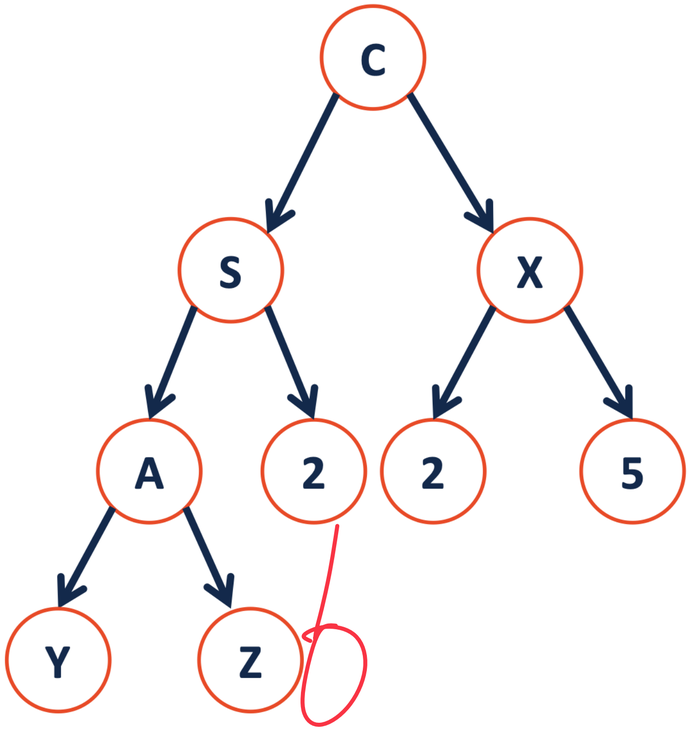
Is every **complete** tree **full**?



Tree Property: complete

Is every **full** tree **complete**?

Is every **complete** tree **full**?



Full : No X

Complete : Yes ✓

Tree ADT

insert, inserts an element to the tree.

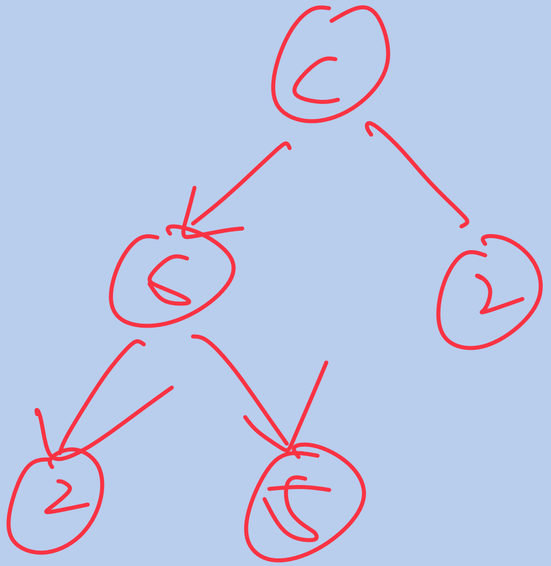
remove, removes an element from the tree.

traverse,

BinaryTree.h

```
 1  #ifndef BINARYTREE_H
 2  #define BINARYTREE_H
 3
 4  template <class T>
 5  class BinaryTree {
 6    public:
 7      /* ... */
 8
 9    private:
10
11
12
13
14
15
16
17
18
19
20  };
21
22  #endif
```

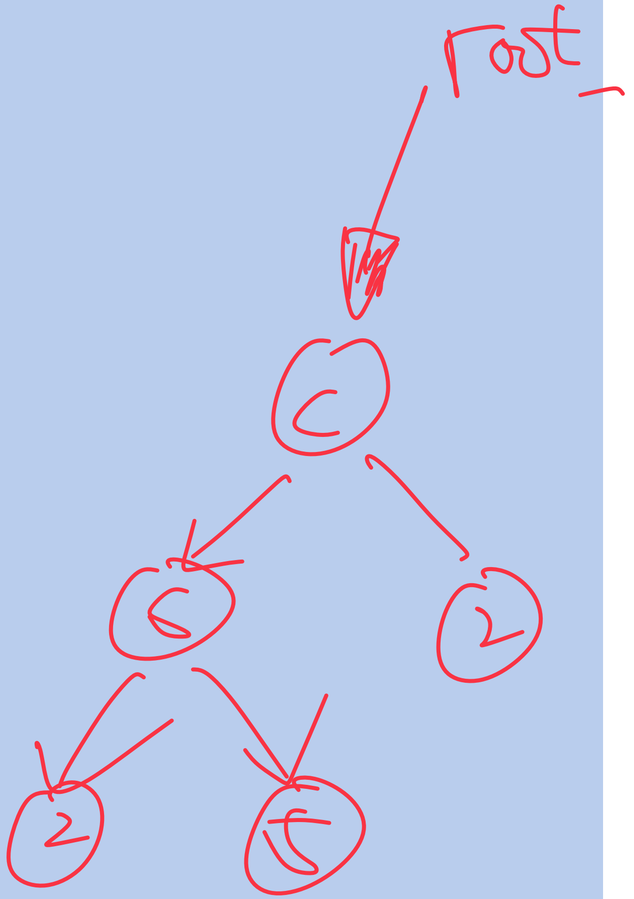
struct TreeNode {
    T data;
    TreeNode *left, *right;
};



BinaryTree.h

```
 1  #ifndef BINARYTREE_H
 2  #define BINARYTREE_H
 3
 4  template <class T>
 5  class BinaryTree {
 6    public:
 7      /* ... */
 8
 9    private:
10
11
12
13
14
15
16
17
18
19
20  };
21
22  #endif
```

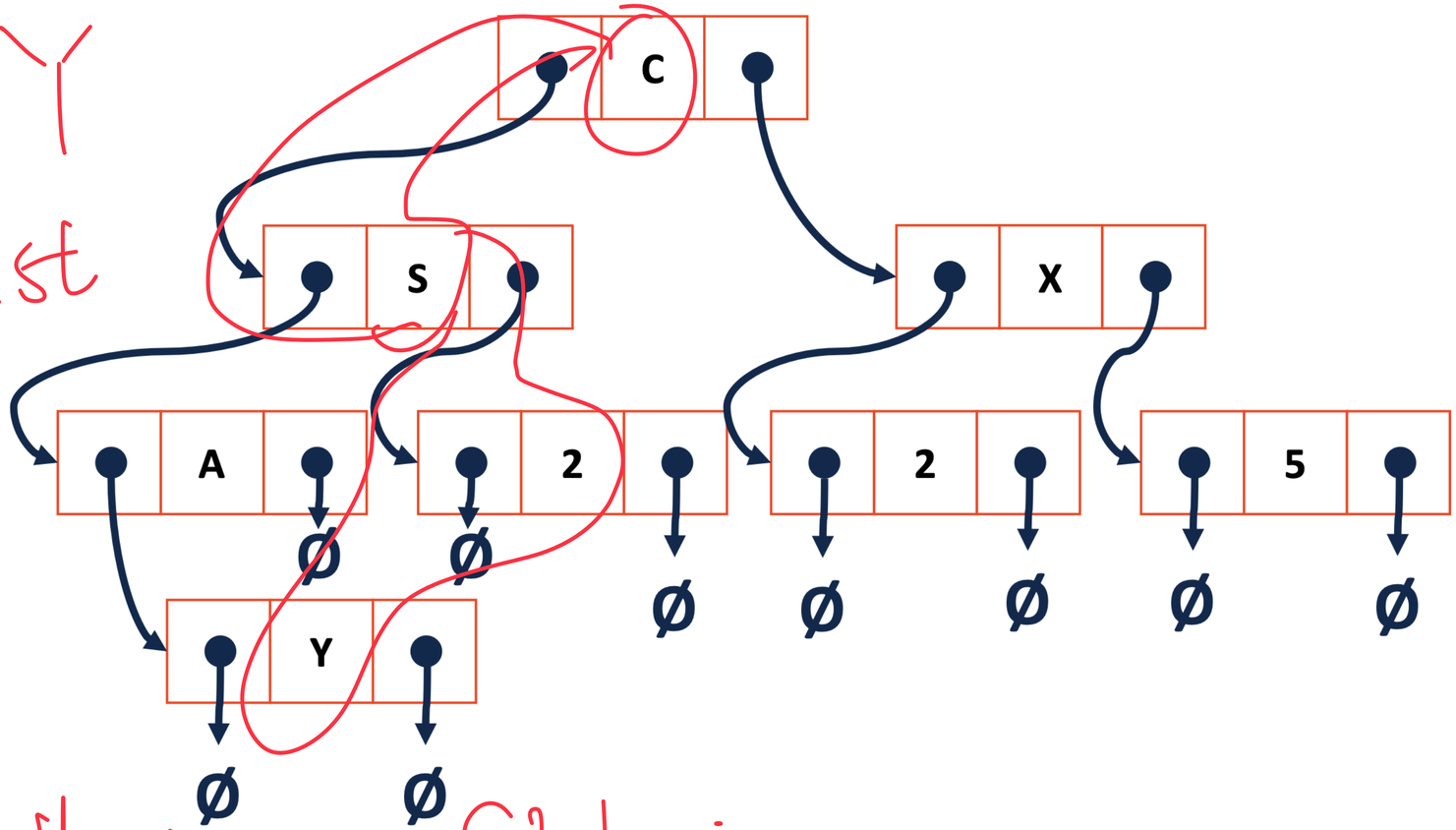
struct TreeNode {
T data;
TreeNode *left, *right;
};
TreeNode *root_;



Trees aren't new:

C → S → 2 → Y

Singly linked list

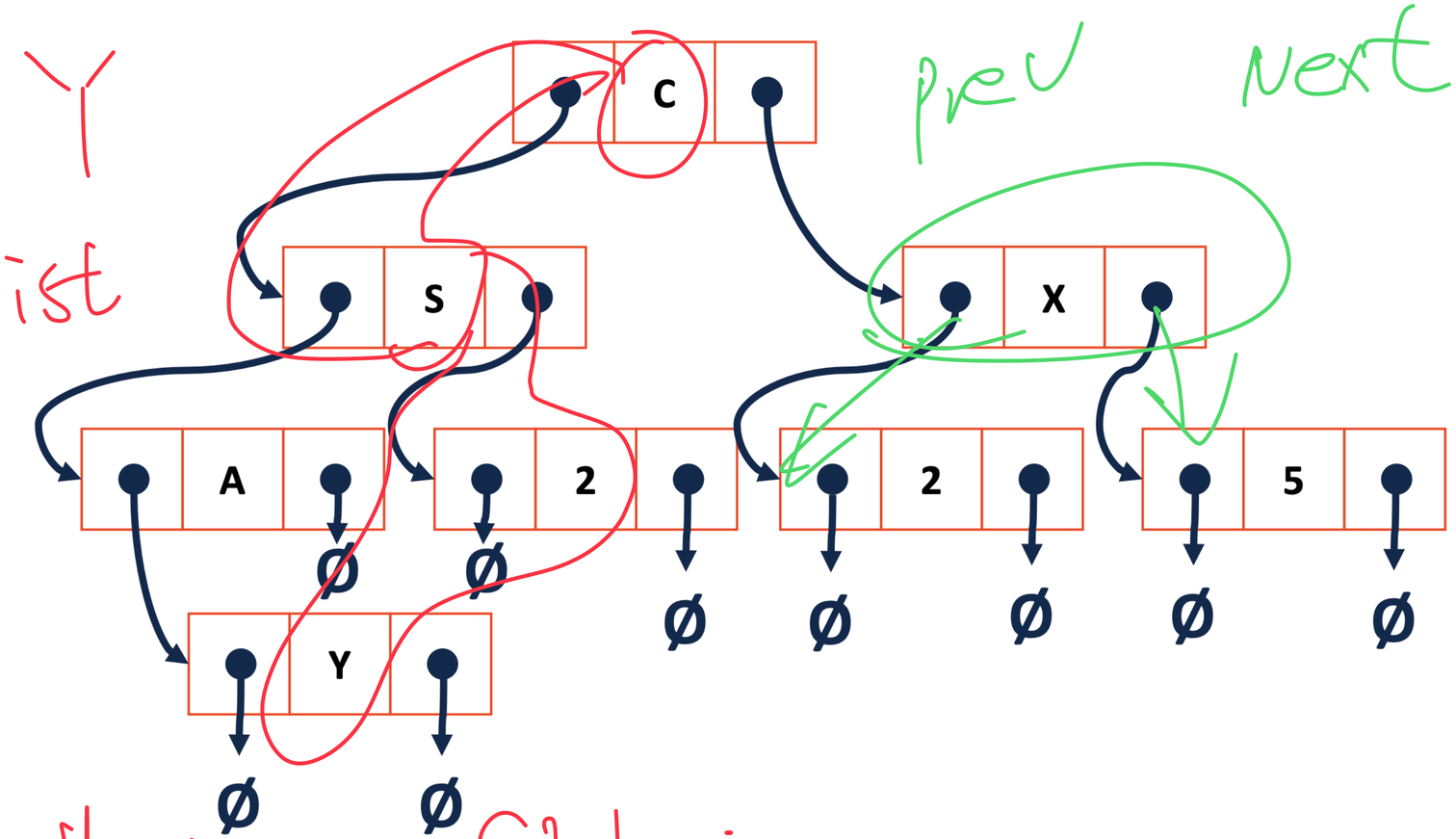


- Every tree path is a SLL :

Trees aren't new:

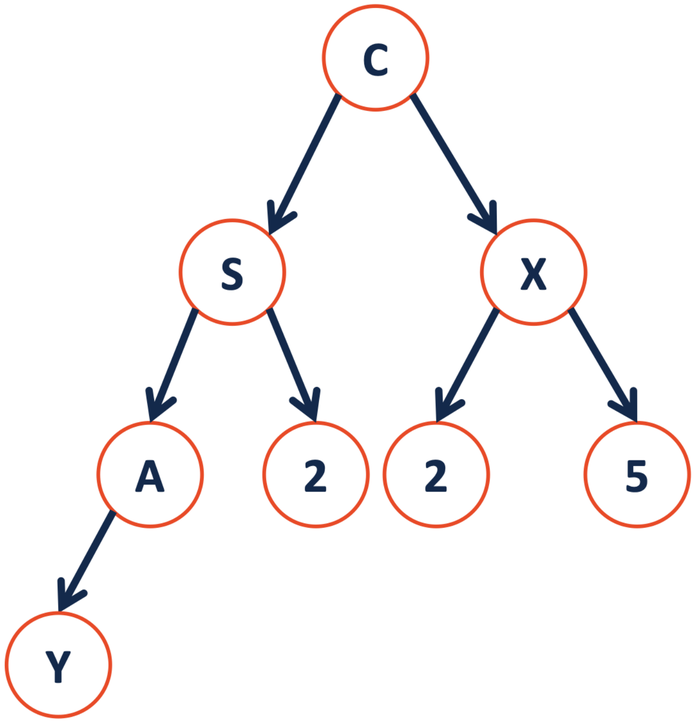
C → S → 2 → Y

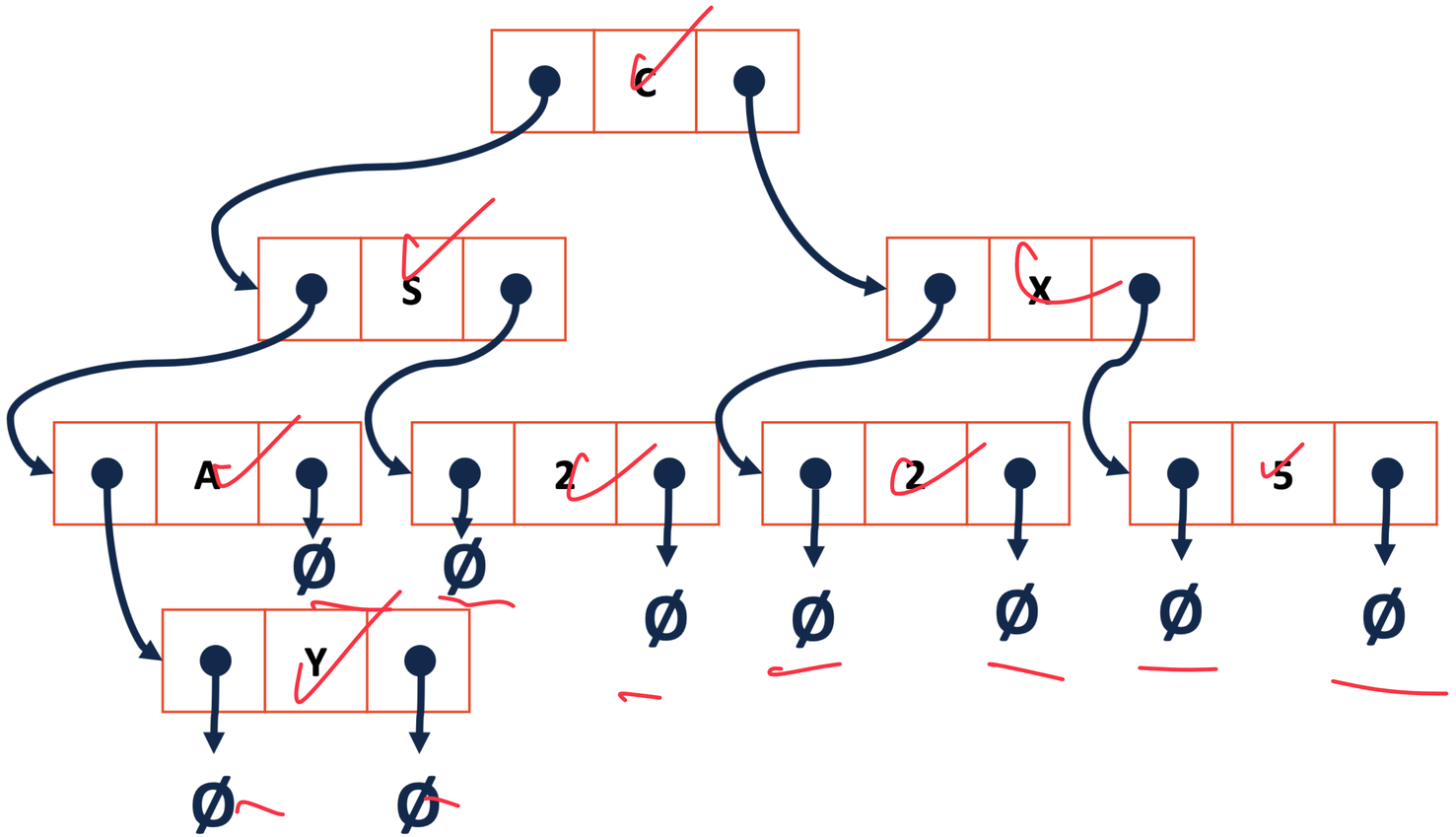
Singly linked list



- Every tree path is a SLL;
- Every tree Node is a DLL struct.

Trees aren't new:





count the Null : 9.

Count the Nodes : 8

How many NULLs?

Theorem: If there are **n** data items in our representation of a binary tree, then there are ____n+1____ NULL pointers.

① $T = \{\}$; $|T| = 0$. root → ϕ.

The tree contains 1 null pointer!

How many NULLs?

Theorem: If there are **n** data items in our representation of a binary tree, then there are ____n+1____ NULL pointers.

① $T = \{\}$; $|T| = 0$, root → ∅.

The tree contains 1 null pointer!

② $T = \{r, T_L, T_R\}$, such that: $|T| = n$, $|T_L| = a$, $|T_R| = b$

∀ Trees with $j < n$ elements, there are $j+1$ Null pointers

How many NULLs?

Theorem: If there are **n** data items in our representation of a binary tree, then there are ___n+1___ NULL pointers.

(2). $T = \{r, T_L, T_R\}$, such that:

$\forall$ Trees with $j < n$ elements, there

$|T| = n$, $|T_L| = a$, $|T_R| = b$

$\therefore$ T_L : $a+1$ Null pointers

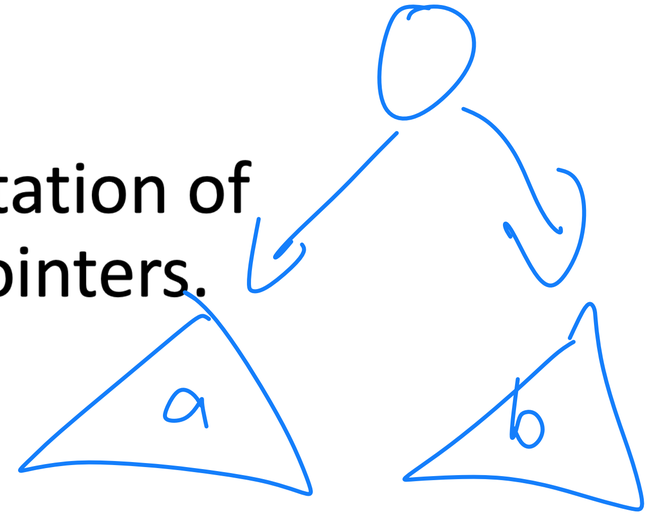
T_R : $b+1$ Null pointers.

are $j+1$ Null pointers.

T : $a+1+b+1$ Null Pointers.

How many NULLs?

Theorem: If there are **n** data items in our representation of a binary tree, then there are ___n+1___ NULL pointers.



② $T = \{r, T_L, T_R\}$, such that:

$\forall$ Trees with $j < n$ elements, there

$|T| = n$, $|T_L| = a$, $|T_R| = b$

$\therefore T_L$: $a+1$ Null pointers

T_R : $b+1$ Null pointers.

are $j+1$ Null pointers.

T : $a+1+b+1$ Null Pointers.

$n = a+b+1$

$\Rightarrow T$: n elements

$n+1$ Null Pointers

CS 225 – Things To Be Doing

MP3: Available now!

Up to +7 extra for submission by next Monday!

Lab: lab_quacks is due on Sunday!